KVFold: Expanding Cache Capacity for LLM Agents with Lossless Compression

Working draft
Author information to be supplied

***Abstract*—LLM agents often reuse long prompt prefixes as they continue a task or resume a session. Caching the corresponding key/value (KV) states avoids repeated prefill, but cache capacity limits how many requests can run concurrently and how much history can be retained. KVFold uses lossless exponent coding to fit more KV into GPU memory and external storage. Pool mode stores FragPack records on the GPU, decoding them into BF16 for FlashAttention-3 during prefill and within a fused attention kernel during decode. Tier mode uses a separate record format in LMCache and leaves the GPU cache layout and attention implementation unchanged. In recorded AgentX runs with a fixed 54 GiB KV budget per GPU, pool mode increases usable capacity from 1.18M to 1.59M tokens. Visitors can extend and edit a coding-agent prompt to inspect prefix reuse and KV reconstruction, then explore recorded results showing how the additional capacity affects concurrent requests and returning sessions.**

I. Introduction

Consider a coding agent investigating a timeout. Its prompt includes repository context, a bug report, and the investigation history. When the user follows the diagnosis with a request for a regression test, most of the prompt remains unchanged. Prefix caching lets the engine reuse the keys and values already computed for this shared context, avoiding repeated prefill. vLLM stores KV in paged blocks [1], while LMCache retains reusable prefixes in DRAM and NVMe across requests [2].

For Qwen3-30B-A3B, each cached token requires 96 KiB of BF16 KV across all layers. With many concurrent sessions, requests may have to wait for free GPU blocks even when their prefixes are available in external storage. Moving inactive histories to external tiers relieves GPU memory pressure, but those tiers can also fill up. When a session resumes after its prefix has been evicted, the engine must recompute that prefix. Cache capacity thus limits both request concurrency and the amount of history available for reuse.

KVFold uses lossless compression to fit more KV into the available GPU memory and external storage. The codec preserves each BF16 value's low byte, assigns short codes to frequent high-byte values, and uses escapes for the remaining values. Unlike quantization methods such as KIVI [3] or lossy context codecs such as CacheGen [4], it reconstructs the original KV bits. The main *pool mode* stores completed GPU blocks as FragPack records. During prefill, KVFold gathers and decodes the context's KV into a BF16 workspace for FlashAttention-3 (FA3) [5]. During decode, it reconstructs operands within a fused attention kernel. An alternative *tier mode* stores compressed KV in LMCache while retaining the native GPU cache layout and attention implementation. Both modes share the coding principle and offline calibration, but use different record formats for attention and external storage.

The demonstration follows the agent as it continues a task, runs alongside other sessions, and resumes an earlier investigation. Interactive requests show prefix reuse, external restoration, and KV byte checks. Recorded AgentX workloads [6] illustrate the effect at scale: with a 54 GiB KV budget per GPU, pool mode holds 1.59M tokens compared with 1.18M for the uncompressed baseline. With admission capped at 12 requests, mean engine time to first token (TTFT) falls from 5.16 to 1.94 s, excluding time spent waiting at the proxy.

II. System Overview

KVFold integrates with vLLM 0.30 and the multiprocess cache server in LMCache 0.5.6 (Figure 1). vLLM schedules requests and allocates 128-token blocks. LMCache handles external lookup, LRU eviction, write-through, and prefetch.

A. Compressing closed blocks in the GPU pool

New KV is written to a BF16 staging buffer as each block fills. At each layer, the backend writes the new values, computes attention, and encodes any blocks that are now complete. These records remain immutable in the layer's compressed pool. Incomplete blocks stay in staging so that later tokens can be appended.

A FragPack record stores either K or V for one block, layer, and KV head: 128 tokens with 128 dimensions each, or 32 KiB in BF16. Seven frequent high-byte values have direct three-bit codes. Other values use a four-bit escape selector to choose among 15 additional values or fall back to a raw high byte. K and V use separate tables, calibrated for each layer and KV head. Low bytes are stored unchanged. The dense streams follow the attention kernel's fragment order, placing scalars in the order each thread needs them for computation (Figure 2).

A block's K and V records share a page within each layer. For four KV heads, a page occupies 190,464 B instead of 262,144 B, allowing the same GPU budget to hold 1.376 times as many pages before accounting for auxiliary backend buffers. Records that exceed the encoding limit or do not fit in a shared page are stored uncompressed in overflow slots. The reported

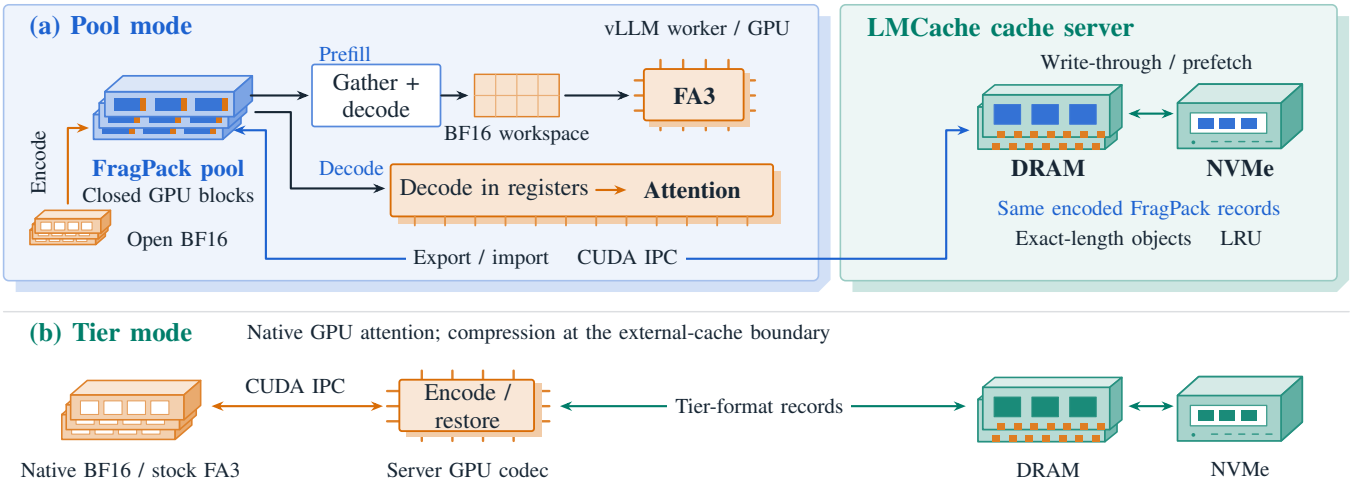


Fig. 1. KVFold's two integration paths. Pool mode stores FragPack records on the GPU and exports the same records to LMCache. Tier mode keeps the native GPU cache layout and encodes and restores KV in the cache server. In both modes, LMCache manages storage in DRAM and NVMe, including eviction, write-through, and prefetch. Complete blocks are encoded; open blocks retain BF16 values for updates and attention.

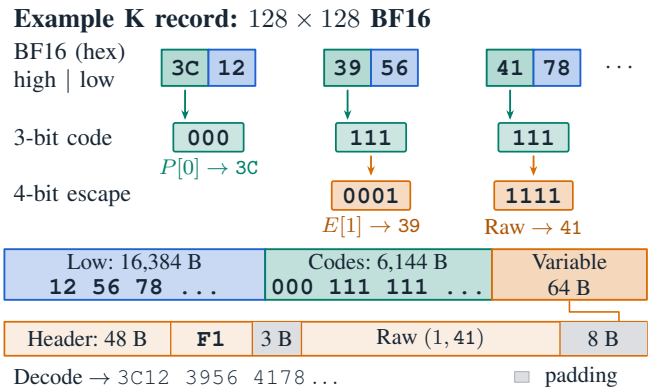


Fig. 2. Example of the FragPack record format using illustrative data (not to scale). The three scalars show direct lookup, an extended-table escape, and a raw escape. Each raw patch stores an escape's position in the escape sequence and its original high byte. The variable region uses 64 B in this example, within the format's 2,048 B limit. Recombining the high and low bytes recovers the original BF16 values exactly.

token capacity is calculated after reserving GPU memory for staging, overflow, and workspaces.

B. Computing attention from compressed KV

Prefill with gather + FA3. When a request extends the context, the adapter gathers the context's KV into a BF16 workspace, decoding compressed records and copying values from open blocks. An unmodified FA3 kernel computes attention from this workspace, which holds the current layer's KV. Completed blocks remain compressed in each layer's pool.

Decode with fused attention. For each generated token, a kernel divides the request's context into page ranges, prefetches encoded records into shared memory, and reconstructs operands in registers for tensor-core attention. A final kernel combines partial softmax states across the ranges. This path avoids writing the decoded context to GPU global memory and supports decode-only CUDA graph replay. The backend is loaded through vLLM's plugin interface and requires no changes to vLLM's source.

Both paths reconstruct the original KV bits exactly. Because the fused kernel may use a different reduction order from FA3, the console displays generated text separately from its KV byte checks.

C. Storing KV in LMCache

Once a block is complete, pool mode can save its attention records to external storage for later reuse. An export kernel packs the records from all layers into a relocatable blob that includes their dimensions, table identity, and lengths. LMCache stores the blob at its actual length in DRAM and NVMe. On import, the records are restored to GPU pages and their record tables are updated. If validation fails, the load is rejected and vLLM recomputes the KV. The cache server accesses worker pools through CUDA IPC and launches export and import kernels on the corresponding GPU, with one synchronization per load request. Each external chunk corresponds to one 128-token engine block.

For deployments that keep the native GPU cache layout and attention implementation, *tier mode* compresses only the KV in external storage. Each chunk contains one block's KV across all layers. Both modes share the calibration procedure, with separate tables for K and V at each layer and KV head. Tier records use the first 14 symbols of each table, seven primary and seven secondary, while FragPack uses all 22. Values are stored in token order. Escapes from the three-bit primary code lead to a second three-bit code and, if needed, a literal high byte. Low bytes remain unchanged. FragPack instead uses fragment order and four-bit escape selectors. An index for each group of eight 256-scalar tiles stores base escape counts and relative offsets to locate the escape streams. Codebooks are fixed at startup and identified by a digest of the codec tables. Chunks whose encoded size exceeds the raw storage limit are stored uncompressed.

Each tensor-parallel (TP) rank stores its own exact-length objects, aligned to 4 KiB for NVMe direct I/O. After the step that produced the KV finishes, the server encodes complete chunks directly from native pages into a bounded GPU store pool. A separate worker synchronizes to read the encoded lengths, allocates DRAM, copies the records, and commits them, leaving the load thread free to serve requests. During a load, compressed records pass through a rotating set of buffers and are decoded into destination pages. Raw records and partially shared leading chunks are first restored into a scratch buffer, then scattered into the destination pages. CUDA events coordinate copies, decoding, and page reuse without host waits on the load path. LMCache uses the stored lengths when prefetching objects from NVMe. Allocating space for each object's encoded length lets the same budget retain more session history.

III. Demonstration

Visitors follow the timeout investigation through prefix reuse, GPU pressure, and returning sessions (Figure 3). The console identifies live and recorded data, model, TP degree, KV type, mode, and budgets. Modes and codebooks are fixed at startup; selectors change the displayed records. Offline, the saved session remains viewable with request controls disabled.

Setup. The interactive session uses Qwen3-4B-Instruct-2507 with BF16 weights and KV, TP 1, and one H800 PCIe GPU. It uses eager execution in pool mode, with 128-token blocks, a 4 GiB GPU KV budget, and 1 GiB of LMCache DRAM.

The recorded capacity experiments used Qwen3-30B-A3B-Instruct with FP8 weights and BF16 KV, TP 2, and two H800 SXM GPUs connected over PCIe. vLLM used 128-token blocks and 8,192-token chunked prefill. AgentX harness 1.0.7 replayed `cc_traces_weka_062126_256k` with seed 42 and recorded think times. P counts live root sessions; a FIFO proxy admits at most S concurrent requests. Measurements covered 1,800 seconds after priming and ten warmup requests, excluding requests without a first token by the window's end. *Engine TTFT* excludes proxy wait. *Goodput* counts requests/minute with engine TTFT at most 2 s and inter-token latency (ITL) at most 40 ms.

These measurements and output checks predate the shared calibration and grouped index described in Section II. The recorded tier runs used per-plane codebooks and the earlier tile index.

A. Step 1: extend a task and inspect prefix reuse

With repository context and a bug report loaded, visitors use *Build* to submit "Diagnose the timeout failure." They can then *Append* the request "Write a regression test." A prefix strip summarizes reused and newly computed tokens alongside the engine's cached-token count, TTFT, and continuation. *Edit early* changes an early token, limiting reuse to complete 128-token blocks before the edit.

Restore waits for external saves, resets the GPU prefix cache, and replays the original prompt. The console reports the source and token counts: in the captured session, LMCache restored 512 tokens from CPU DRAM.

TABLE I
GPU capacity and performance in the recorded runs: 54 GiB KV per GPU, $P = 64$, 400 GiB DRAM, and 2 TiB NVMe. Wait denotes the mean number of requests waiting in vLLM; TTFT is mean engine TTFT in seconds.

S	Mode	KV tokens	Wait	TTFT	Goodput
8	Raw	1.18M	0.3	1.69	15.1
8	Pool	1.59M	0.2	1.34	16.2
12	Raw	1.18M	1.5	5.16	10.6
12	Pool	1.59M	0.4	1.94	14.9

The inspector compares decoded KV from an already encoded block before and after export through host memory and import into an independent GPU scratch store. All layers of the captured 128-token block matched byte for byte. It displays sizes, checksums, and mismatch counts. Separate CPU reference samples exercise escapes and raw fallback.

A separate historical output check found identical greedy outputs for 192 repeat-trace requests in tier and raw-tier modes with matching execution paths.

B. Step 2: explore GPU capacity limits

Visitors switch between $S = 8$ and $S = 12$ to compare raw and pool modes under equal GPU KV budgets (Table I). The external tiers retain the working set, with cached-token share fixed at 0.94. These runs examine GPU pressure when prefixes are already cached.

Pool mode reserves 1.3 GiB within the KV budget for staging, overflow, and the prefill workspace. Cache-server CUDA contexts and transfer buffers are accounted for separately; codec buffers occupy 1.0 GiB per GPU outside vLLM's budget.

At $S = 12$, pool mode keeps TTFT close to the raw baseline at $S = 8$, while reducing waiting requests and increasing goodput at the same admission cap. The console places these changes beside usable KV capacity.

C. Step 3: revisit sessions under storage pressure

Visitors switch between 640 GiB and 2 TiB NVMe to compare retained history, stored bytes, TTFT, and goodput (Table II). Raw and tier modes use the same native GPU KV layout, FA3, and LMCache LRU, write-through, and prefetch policies; the allocated record sizes differ.

With 640 GiB of NVMe, tier mode retains more prefixes for returning sessions (Table II). Its compressed-to-raw size ratio of 0.716 allows the same storage capacity to hold about 1.4 times as much KV. With 2 TiB, the uncompressed working set already fits, so the cached-token shares converge. In this case, compression adds overhead: input throughput is 26.9 thousand tokens/s in tier mode, compared with 27.5 for raw storage, a 2.2% reduction.

A separate experiment replayed 48 prompts with one request in flight at a time to compare DRAM hits with NVMe hits.

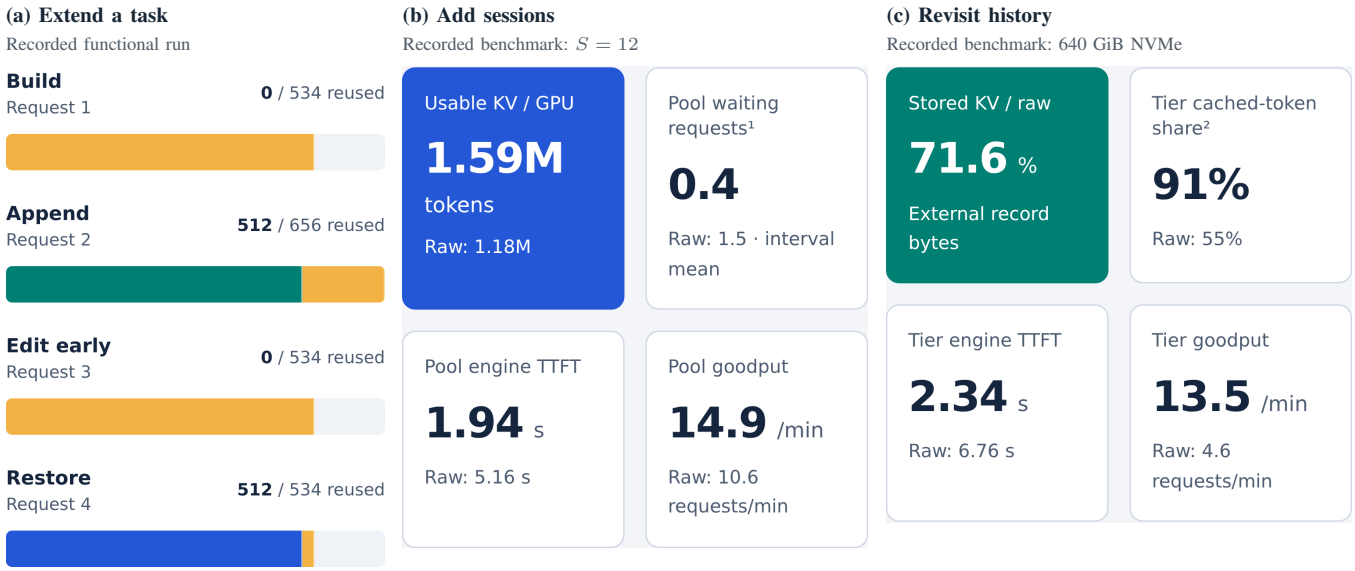


Fig. 3. Screenshots of the implemented console. Left: a saved Qwen3-4B prefix-reuse session. Center and right: recorded GPU-capacity and external-capacity comparisons from Tables I and II, measured separately with Qwen3-30B-A3B. Superscripts denote [1]mean waiting requests and [2]the combined GPU/external cached-token share.

TABLE II
EXTERNAL CACHE CAPACITY AND PERFORMANCE IN THE RECORDED RUNS: $P = 64$, $S = 8$, AND 400 GIB DRAM. CACHED DENOTES THE SHARE OF PROMPT TOKENS SERVED FROM THE GPU OR EXTERNAL CACHE; TTFT IS MEAN ENGINE TTFT IN SECONDS.

NVMe	Mode	Cached	TTFT	Goodput
640 GiB	Raw	0.55	6.76	4.6
640 GiB	Tier	0.91	2.34	13.5
2 TiB	Raw	0.94	1.69	15.1
2 TiB	Tier	0.94	1.85	15.4

The KV occupies 261 GiB uncompressed and 188 GiB compressed. With 220 GiB of DRAM, the compressed working set fits entirely in memory, while some uncompressed prefixes must be loaded from NVMe. Across the recorded 8K–128K prompts, mean warm TTFT for raw storage is 1.63–1.68 times that of tier mode. Both working sets fit in 400 GiB of DRAM, where the latency gap narrows. Selecting this trace displays static comparisons of recorded residency and latency. Showing a transfer timeline would require additional time-series measurements.

IV. CONCLUSION

KVFold fits more KV into GPU memory and external cache tiers. Visitors inspect prefix reuse, restore KV from LMCache, and check a model block before and after transfer. Recorded comparisons show how the additional capacity affects request queues and history reuse under fixed budgets.

ACKNOWLEDGMENT

OpenAI Codex assisted with drafting and figure preparation. Experimental values were taken from the authors' recorded results.

REFERENCES

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[2] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," *arXiv preprint arXiv:2510.09665*, 2025. [Online]. Available: https://arxiv.org/abs/2510.09665

[3] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 32 332–32 344. [Online]. Available: https://proceedings.mlr.press/v235/liu24bz.html

[4] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024. [Online]. Available: https://arxiv.org/abs/2310.07240

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," *arXiv preprint arXiv:2407.08608*, 2024. [Online]. Available: https://arxiv.org/abs/2407.08608

[6] SemiAnalysis, "InferenceX AgentX: Methodology and datasets," https://inferencex.semianalysis.com/agentx, 2026.